What Is an Exact Memory Footprint Worth to a Scheduler? Measured Footprints and Job Scheduling on Disaggregated Memory

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A scheduler for a cluster with disaggregated memory can pack jobs only as tightly as its knowledge of each job's memory footprint allows. We measure that knowledge for 25 programs at three inputs each, two LLM inference engines among them, with the full Pin trace as the truth, MemPrint's windowed and source-only estimates, Massif, DHAT, heaptrack, the gperftools heap profiler and sampled RSS, and we replay the measured footprints through DMSched-Sim, a simulator of 32 nodes with local DRAM and a shared 1 TB pool, in which only the footprint oracle changes. Knowing a job's footprint curve rather than its peak is where most of the gain lies: at 80% offered load the exact curve gives a mean response time 26.4× shorter than the user's request and 5.3× shorter than the exact peak, and it needs 5.0 TB of memory for a service level that the exact peak meets with 6.0 TB and the request with 11.0 TB. Among measured estimators, MemPrint windowed at 5% is the only one that beats the exact peak (389 against 448 minutes), at 11 times the native run time against 227 times for the full trace. The heap profilers report the right peak for most programs but read GCC's cc1 at 0.14 to 0.16 times its peak, llama2.c, whose weights are mapped from a file, at 0.17 times and darknet at 1.97 times, so no margin makes them safe without over-reserving every other job; with the LLM engines in the corpus, the calibrated margins of heaptrack and gperftools exceed twice their estimates. On a stream of LLM jobs alone, the heap profilers fail because the weights are outside the heap, and MemPrint's curves failed because they saw the weights only in their windows; tracking the program's file mappings between windows fixes the latter (386 instead of 4,134 minutes on the LLM stream, and 327 instead of 389 on the full corpus, where every MemPrint curve then beats the exact peak). Curves built from DHAT's three recorded points do worse than DHAT's peak under every margin. When the truth is the resident set on wall-clock time, every measured curve, MemPrint's included, needs about 60% headroom and peaks do better than curves. Placing MemPrint's curve on the native clock, through hardware counts of the native run's memory references, and at the level of an earlier run's RSS cuts that headroom to 17% and its response time from 1,873 to 569 minutes, and against the resident set without code it matches the exact peak (422 against 429 minutes); a curve still gains nothing over a peak there, because the allocator keeps freed memory resident.

1 Introduction

A batch scheduler places a job only after it decides how much memory to reserve for it, and on a cluster with disaggregated memory that decision also sets how much of the shared pool the job takes away from other jobs. Disaggregated memory attaches a pool of memory, over CXL or RDMA, that every node can use in addition to its local DRAM [11, 22]. The pool lets a cluster provision less local memory per node, but only if the scheduler can pack jobs tightly, and it can pack them only as tightly as its knowledge of each job's memory allows. Today that knowledge is

weak. Peng et al. found that over 90% of LLNL jobs use under 15% of a node's memory [15], and Zacarias et al. showed that a uniform 60% overestimate of memory demand cuts the throughput of a disaggregated system by about 25% and raises the mean response time about five times [23].

Two kinds of knowledge are possible. A *peak* is one number per job, held for the whole run. A *footprint curve* gives the memory the job needs at every point of its run, so a scheduler can let two jobs whose peaks do not overlap share the same memory. The usual sources of either are the user's request, a heap profiler such as Massif [18], DHAT [19], heaptrack [20] or the gperftools heap profiler [8], the resident set size (RSS) of an earlier run, a learned predictor [11], or a monitor that resizes a job's allocation while it runs [24]. MemPrint [14] adds two more: a footprint curve measured by instrumenting a sample of a run's memory accesses with Pin [13], and a footprint predicted from the program's source code before it runs.

However, the simulators that study scheduling on disaggregated memory either model one run in cycle detail with no scheduler [17, 3, 4, 21], or treat a job's memory as one declared peak with a synthetic error [23, 22]. None feeds the scheduler the footprints that real profilers report, and none compares those with a footprint curve known before the job starts. It is therefore unclear how much better a scheduler does with an exact footprint than with the tools that exist, and which property of the footprint (its peak, its shape over time, or how safely it bounds the truth) the gain comes from.

In this work, we measure every source of footprint knowledge listed above on the same 25 programs and replay the results through one scheduler, with everything but the footprint oracle held fixed. We make the following contributions:

- We build a corpus of footprint curves for 25 programs at three inputs each, HPC kernels, interpreters, a compiler and two LLM inference engines among them, from the full Pin trace (the truth), MemPrint's windowed and static estimates, Massif, DHAT, heaptrack, gperftools and sampled RSS, together with what each method costs (Section 4).

- We build DMSched-Sim, a cluster simulator with local memory and a shared pool, an EASY-backfilling scheduler that admits a job only if its whole reservation curve fits, a remote-memory slowdown model and OOM kills (Section 3).

- We show how much of the gain an exact footprint brings comes from knowing the curve rather than the peak, how close each measured estimator gets, how much memory each needs for the same service, and what each estimator's cost buys (Section 5).

- We evaluate curves derived from DHAT, which records no time series, and show why its three recorded points do not substitute for a measured curve (Section 6).

- We schedule a stream of LLM inference jobs alone, whose memory is mostly model weights mapped from a file, show which estimators survive it (Section 5.6), and extend MemPrint's windowed mode to track file mappings between windows, which removes its failure there (Section 5.7).

2 Background and related work

Disaggregated memory simulators. DRackSim [17], CXL-ClusterSim [3], CXL-DMSim [4] and CXLMemSim [21] model the timing of accesses to pooled memory for one run, in cycle detail or from a trace, and none of them has a job scheduler. The BSC Slurm simulator extended by Zacarias et al. schedules real job traces on a disaggregated system [22], but it represents a job's

memory as one peak, and its study of accuracy multiplies that peak by a synthetic overestimate [23]. Their dynamic provisioning study resizes each job's allocation every five minutes to its use, read from usage traces, and fails and restarts a job whose growth does not fit [24]. That is a monitor that acts while the job runs, not knowledge available before it starts, and we evaluate it as a baseline. Boezennec et al. study the allocation of disaggregated memory to jobs with piecewise-constant memory phases [2], CFM schedules jobs against per-job slowdown curves measured offline [1], and Pond predicts the untouched memory of virtual machines with a gradient-boosted model, places it in a CXL pool and corrects overpredictions with a QoS monitor [11]. We evaluate it as a baseline as well. KS+ predicts the memory of workflow tasks over time and reduces wasted GB-seconds by 51% against peak-only prediction [10], but it has no cluster scheduler or memory pool.

Footprint estimators. Massif samples the heap at up to 100 snapshots by default, on a time axis of instructions or of milliseconds under Valgrind [18], and with `-pages-as-heap` it counts mapped pages instead of heap blocks. DHAT reports the heap at its global peak (t-gmax) and at exit, and the time of the peak, but no time series [19]. heaptrack records every allocation and can export a heap curve [20], and the gperftools heap profiler writes a dump each time the heap grows by a set amount [8]. All four count requested heap bytes, so they miss memory a program maps itself and count blocks that are allocated but never touched. MemPrint counts the bytes a program actually touches, from a sample of its memory accesses [14]. Its windowed mode watches accesses only in short windows (1% or 5% of the run) and tracks allocations and page residency in between, and its static mode predicts the footprint from the source code with an interpreter that counts the references a build of the program makes.

3 Simulator

DMSched-Sim replays one stream of jobs once per footprint oracle, so that any difference between two runs comes from what the scheduler knows about memory. It is a time-slotted simulator (one slot is one minute) written in Python (`analysis/memprint/sched/`).

System model. The cluster has 32 nodes, each with 32 cores and 128 GB of local DRAM, and one shared pool of 1 TB (one sixth of all memory). A job runs on one node. Its memory is placed in the node's free local memory first, and the rest goes to the pool.

Jobs. A job has an arrival time, a core count, a base runtime (its runtime with all memory local), a true footprint curve $a(p)$ over its progress $p \in [0, 1]$, and a reservation curve $r(p)$ set by the oracle. Curves are defined over progress rather than time, so a job slowed by remote memory stretches its curve in wall-clock time.

Scheduler. The scheduler is first come, first served with EASY backfilling [12] over a window of 32 queued jobs. Each node keeps a timeline of committed local memory and cores, and the pool keeps one of committed memory. A job starts on a node only if, at every slot from now to its projected end, the part of its reservation that does not fit in the node's free local memory fits in the free pool. With a time-varying reservation, two jobs whose peaks do not overlap can therefore share the same memory. When the head of the queue cannot start, the scheduler reserves its earliest feasible start, and a later job may start early only if it fits beside that reservation. Among the nodes that fit, the scheduler picks the one with the least pool use and then the least free local memory left over (best fit).

Remote-memory slowdown. Following the linear models of CFM [1] and Boezennec et al. [2], a job's runtime is stretched by the share of its resident GB-minutes that lie above its local reservation:

$$T = T_{\text{base}}\,(1 + \beta f_{\text{remote}}), \qquad f_{\text{remote}} = \frac{\sum_t \max\bigl(0,\, a(t) - \ell(t)\bigr)}{\sum_t a(t)}, \tag{1}$$

where $\ell(t)$ is the local part of the reservation and $\beta = 0.5$, so a job whose memory is entirely remote runs 1.5 times as long. Because the runtime depends on the placement and the placement depends on the runtime, the two are solved together by fixed-point iteration when the job is admitted. The scheduler ranks candidate nodes on the job's base runtime and then solves the stretch on the best node, falling back to the next node if the stretched job no longer fits.

OOM kills. If the true footprint exceeds the reservation at any slot, the job is killed there, its work is lost, and it is requeued at the head of the queue with its reservation multiplied by 1.5, as a batch system that raises a job's memory limit after an out-of-memory failure would do. A reactive oracle (Section 3.3) is the exception: its growth past the reservation is granted from free local memory or the pool while either has room, and the job is killed only when neither does.

Metrics. We report the mean response time (completion minus arrival), the mean bounded slowdown (response over $\max(T_{\text{base}}, 10\,\text{min})$), the number of OOM kills, and the reserved and the actually resident memory as shares of all memory. To leave out the warm-up and the drain of a finite stream, every metric is taken over the jobs that arrive between the 10th and the 90th percentile of arrival times, and the memory shares are averaged over that period.

Validation of the simulator. Fifteen unit tests (`tests/sched/test_sim.py`) check that two jobs with non-overlapping peaks share a node under their curves but run one after the other under their peaks, that an under-reservation is killed where the footprint passes it and requeued, that backfilling never delays the head of the queue, that a fully remote job runs $1+\beta$ times as long, that elastic growth is granted only from free memory, that Pond's untouched part goes to the pool and its QoS monitor shortens an overpredicted job's stretch, and that a dynamic resize that no longer fits restarts the job. On the synthetic stream of the earlier prototype (3,000 jobs, five footprint shapes, a 60 GB median heap) the simulator reproduces the prototype's ordering at saturation: the exact curve sustains 101 jobs per hour, the exact peak 16% fewer, a Massif-style curve 37% fewer and a DHAT-style peak 57% fewer.

3.1 Job streams

The main experiments use a controlled stream of 3,000 jobs in which memory is the contended resource. Each job has 1 to 16 cores, a lognormal base runtime with a median of 30 minutes clipped to 5 to 720 minutes, and a lognormal memory request with a median of 100 GB. What a job really uses is its request multiplied by a ratio $u \le 1$ taken from the LANL CM5 log of the Parallel Workloads Archive [5], the archive log that records both requested and used memory (median $u = 0.79$ over its 94,335 jobs with both fields). The shape of the job's footprint, and what every estimator reports for it, come from one corpus program at its largest input (Section 4), chosen at random. All of that program's curves are scaled by the factor that puts the truth's peak at the job's peak, so each estimator keeps its own over- or under-estimate. Arrivals are spread evenly in random order over a span set so that the *offered load*, the resident GB-minutes arriving per minute over the cluster's total memory, takes the values 0.4, 0.6, 0.8, 1.0 and 1.2. Each point is three seeds.

Table 1: Footprint oracles. "Curve" oracles follow a measured footprint over the job's progress; "peak" oracles hold one value for the whole run.

Oracle	Kind	Source
Exact curve	curve	the truth, at full resolution (the upper bound)
Exact, 20 windows	curve	the truth, coarsened as the measured curves are
Exact peak	peak	the truth's peak
MemPrint windowed 1%, 5%	curve	MemPrint watching 1% or 5% of the run
MemPrint pages only	curve	MemPrint page residency, nothing watched
MemPrint static	curve	predicted from source before any run
Massif curve, pages-as-heap	curve	Massif snapshots (heap, or mapped pages), instruction clock
Massif, ms clock	curve	the same, with time in milliseconds under Valgrind
heaptrack curve	curve	heaptrack's heap over time
DHAT peak	peak	heap at DHAT's global peak (t-gmax)
DHAT curve, hold-then-decay	curve	three points recorded by DHAT (Section 6)
gperftools peak	peak	largest in-use heap of the heap profiler's dumps
RSS peak	peak	peak RSS of an earlier run of the same input
User request	peak	the job's memory request
ML peak	peak	request $\times$ a 0.95-quantile gradient-boosted prediction of u
Reactive resizing	curve	request for 5 minutes, then the last 5 minutes' largest use + headroom
Pond	peak	the request, its predicted untouched part in the pool; QoS monitor
Dynamic provisioning	curve	request, then each 5 minutes the coming 5 minutes' largest use

We also tried the MetaCentrum2 log [5] (2013 to 2015, 5.7 million jobs with a memory request but no used memory). In a window of 3,000 single-node jobs, 1% of the jobs carry 92% of the requested GB-minutes, so the results of a replay depend on a handful of very large jobs. We therefore use the controlled stream for the main results.

3.2 Two truths

MemPrint counts the bytes a program touches, but a kernel charges and OOM-kills a process for its resident pages. The two differ by page granularity, by memory an allocator keeps after a free, and by the runtime's own pages (loader, libraries, MPI, an interpreter). We therefore run every experiment with two truths. With *bytes touched* as the truth, the scheduler must cover the footprint a page-perfect system would need. With *RSS* as the truth, it must cover the resident set sampled every 5 ms. The runtime's own pages do not grow with a job's data, so under the RSS truth only the RSS above the run's smallest sample (1.2 to 9.5 MB across the corpus) is scaled to the job's size.

3.3 Oracles and margins

An oracle turns what is known about a job into its reservation curve. Table 1 lists them. Curve oracles follow a measured curve, coarsened to 20 windows over progress, each holding its largest value and widened to its neighbours, because each method's clock (memory references for MemPrint, instructions or milliseconds under Valgrind for Massif, seconds for RSS) maps to wall-clock progress only approximately. Peak oracles hold one value for the whole run.

The ML oracle follows Pond [11]. It predicts u from the request, the core count, the requested time and the user's and the executable's median u in their history, with a gradient-boosted model at the 0.95 quantile, trained on half of the CM5 users. The jobs it predicts take their u from the other half of the users. The reactive oracle is a simplified form of the dynamic provisioning of

Zacarias et al. [24] that sees only the past. The scheduler is shown its future resizes when it admits the job, which favours this baseline.

Two further baselines follow the two published systems as closely as the simulator allows. The *Pond* oracle [11] reserves the job's whole request, so it never causes an OOM kill, but it keeps only the request minus the predicted untouched memory in local DRAM. The rest, rounded down to whole GB, is placed in the pool, as Pond's zero-core NUMA node does for a virtual machine, even when local memory is free. The untouched share is predicted by the same gradient-boosted model as the ML oracle, at the 0.98 quantile, which corresponds to Pond's target of 98% of jobs within their performance margin. A job that touches more than it was given locally runs on the pool. Five minutes after it first does so, Pond's QoS monitor moves its pool part to local memory, where the node has room. Pond's second model, which places latency-insensitive virtual machines entirely in the pool, needs hardware performance counters that the job streams do not have, and is left out. The *dynamic provisioning* oracle follows the Slurm policy of Zacarias et al. [24]. A job starts with its request. Every five minutes, its allocation is resized to the largest use of the coming five minutes, which their simulator reads from the usage trace. The resize takes local memory first and the pool next, with no headroom. Unlike the reactive oracle, a resize is committed only when it happens, so another job may take the memory first. A resize that does not fit then fails the job, which restarts from the beginning (their fail/restart policy), and we count it with the OOM kills. Its runtime stretch is fixed when it is admitted, from the placement it would get then. Neither oracle uses a margin.

Every estimate needs a margin, because an estimate below the truth causes OOM kills. A curve oracle's margin is headroom added as a share of its own peak, since a curve on another clock can be low where the truth is not, and no factor lifts a zero. A peak oracle's margin multiplies its peak. We use four margin policies: a fixed 10% and 30%, and two *calibrated* margins that a site could set from its own history. For each oracle and program we compute the smallest margin with which the oracle never falls below the truth. A job then receives the 90th percentile (*calibrated 90%*) or the largest value (*calibrated 100%*) of the other programs' margins, so no oracle is tuned on the program it predicts. The request, ML and reactive oracles have no profile to calibrate, and they keep a 10% margin.

4 Footprint corpus

We ran 25 programs at three inputs each, from the smallest to the largest, on a 32-core node with 62 GB of memory (Table 2): four PolyBench kernels [16] (2mm, gemm, jacobi-2d, atax, unoptimised builds at SMALL, MEDIUM and LARGE), miniVite [7] with four threads, the GAP benchmarks bfs, pr, cc and sssp on Kronecker graphs of 2^{16} to 2^{18} vertices, Lua, Python, Perl and SQLite workloads at three sizes, GCC's cc1 compiling three source files at `-O0`, HPCCG, LULESH, NPB IS, CG and MG (classes S, W and A), MiniFE, Quicksilver, Kripke, darknet classifying images with AlexNet (one input), and two LLM inference engines: llama2.c [9] generating 128 tokens from "Once upon a time" with the stories15M, 42M and 110M models, and llama.cpp [6] (b11500, CPU, four threads) completing the same prompt with SmolLM2-135M (32 and 128 tokens) and SmolLM2-360M (64 tokens), quantised to 8 bits, at the model's default 8192-token context. Both engines map their weights from the model file rather than allocating them, and llama.cpp sizes its KV cache for the whole context and clears it with `memset`. The native runs take 0.12 to 14.9 s, and the true peaks at the largest inputs are 15 to 450 MB.

Every (program, input) ran under ten methods: the program alone, with its RSS sampled every 5 ms; the full Pin trace (MemPrint's splitter with frees tracked), which gives the bytes touched over

Table 2: The corpus at each program's largest input: the true peak (bytes touched), the mean over the run as a share of the peak, each method's peak as a share of the true peak, and the time each method takes over the native run.

		Footprint (ratios to the true peak)								Cost (time / native)			
Program	Input	Peak (MB)	Mean/peak	RSS	MemPrint W5	Massif	DHAT	heaptrack	Native (s)	W5	Massif	DHAT	Full trace
2mm	LARGE	37.1	0.96	1.03	1.00	1.00	1.00	1.00	8.6	4.8	5.6	15.2	357
atax	LARGE	32.1	0.87	1.04	1.00	1.00	1.00	1.00	0.1	21.8	5.7	9.8	122
GCC cc1	large	105.5	0.69	1.27	1.06	0.16	0.15	0.15	1.8	100.5	11.0	68.1	206
darknet	alexnet	269.9	0.86	1.07	1.04	1.97	1.97	1.97	3.0	11.8	10.5	45.9	773
GAP bfs	18	72.8	0.36	1.05	1.00	0.99	0.99	0.99	1.5	8.1	3.0	5.7	83
GAP cc	18	72.0	0.36	1.06	1.01	1.01	1.01	1.01	2.2	5.5	2.2	3.9	56
GAP pr	18	72.4	0.40	1.05	1.01	1.00	1.00	1.00	2.8	5.2	2.3	5.3	123
GAP sssp	18	135.8	0.36	0.95	1.00	1.00	1.00	1.00	2.5	5.9	2.4	4.4	116
gemm	LARGE	29.1	1.00	1.04	1.00	1.00	1.00	1.00	6.1	4.4	5.7	15.2	356
HPCCG	64	102.7	0.99	1.03	1.00	1.02	1.02	1.02	1.7	13.3	10.5	24.8	310
jacobi-2d	LARGE	27.1	1.00	1.04	1.00	1.00	1.00	1.00	12.8	3.7	7.3	16.2	275
Kripke	16	274.6	0.88	1.02	1.00	1.00	1.00	1.00	3.4	11.4	4.8	50.7	473
llama2.c	110M	449.7	0.98	1.14	1.00	0.17	0.17	0.17	14.9	3.5	3.6	20.5	41
llama.cpp	135m-n128	352.9	0.89	1.08	1.00	1.05	1.05	1.05	1.5	47.1	137.1	279.6	604
Lua	1	57.1	0.53	1.15	0.94	1.02	0.92	0.92	4.2	14.8	4.3	28.0	161
LULESH	60	151.1	0.62	1.02	1.00	1.00	1.00	1.00	5.5	10.6	7.9	32.2	458
miniVite	32768	15.4	0.51	1.99	1.00	1.55	1.53	1.42	1.8	42.5	22.9	36.9	322
MiniFE	80	256.5	0.98	1.04	1.00	1.03	1.02	1.02	9.2	10.7	6.7	23.8	273
NPB CG	A	27.4	1.00	1.10	1.00	0.99	0.99	0.99	1.9	10.6	7.8	15.5	237
NPB IS	A	69.7	0.81	1.03	1.00	0.99	0.99	0.99	2.4	3.9	2.7	7.0	61
NPB MG	W	58.5	0.98	1.04	1.00	0.99	0.99	0.99	0.5	15.5	4.0	8.8	107
Perl	1	56.1	0.67	1.15	0.97	1.04	0.97	0.96	1.8	30.9	5.2	44.8	145
Python	1	95.9	0.37	1.19	1.04	1.29	1.01	0.14	5.6	17.0	12.5	175.5	272
Quicksilver	100000	42.2	1.00	1.28	1.01	1.02	1.01	1.01	5.5	13.9	6.3	44.5	227
SQLite	1	77.8	0.43	1.06	0.97	1.01	1.01	1.01	1.2	47.7	7.1	39.0	204

time and is the truth; MemPrint windowed at 1% and 5% watched and pages only, with the tool built from the `windowed-next` branch; Massif and Massif with `-pages-as-heap` (Valgrind 3.22, up to 1,000 snapshots), each run twice, with Massif's default clock of instructions executed and with milliseconds under Valgrind; DHAT; heaptrack 1.3.0; and the gperftools 2.18 heap profiler with a dump every 1/50 of the native peak RSS. The MemPrint static predictions come from the C interpreter for PolyBench (a curve over time) and from the C++ interpreter for HPCCG, LULESH and NPB (a whole-run footprint). The NPB predictions are those of the second blind test, made before any trace of NPB existed; the static route does not cover the LLM engines. Of the 874 runs, two failed: the full traces of NPB MG class A and of llama.cpp with the 360M model ran out of memory (the latter at 20.7 GB, in Pin's allocator), so MG is used up to class W and llama.cpp up to the 135M model with 128 tokens. The runs shared the node with up to 15 others (the LLM runs with up to 7), so the costs below are slowdowns under load.

4.1 Accuracy of the peak

Most heap profilers report the right peak for most of these programs, and fail badly on the rest (Figure 1). Specifically, Massif's peak is within 5% of the true peak on 20 of the 25 programs, DHAT and gperftools on 20 and heaptrack on 19, because these programs touch nearly all the heap they allocate. However, the heap profilers read 0.14 to 0.16 times the peak of GCC's cc1, whose garbage collector maps its own pages, and 0.17 times that of llama2.c, whose 110M model's weights are mapped from the file and never pass through the heap; they read 1.97 times the peak of darknet and 1.42 to 1.55 times that of miniVite, which allocate memory they never touch. On llama.cpp they are within 5% (1.04 to 1.05), because its weights' file mapping is offset by a KV cache that it allocates on the heap and clears in full. MemPrint windowed at 5% is within 5% on 23 of the 25 programs (0.94 to 1.06 times the truth). Pages only is within 5% on 21; it estimates the resident pages of heap blocks and writable sections, so it misses the read-only weight mappings and reads

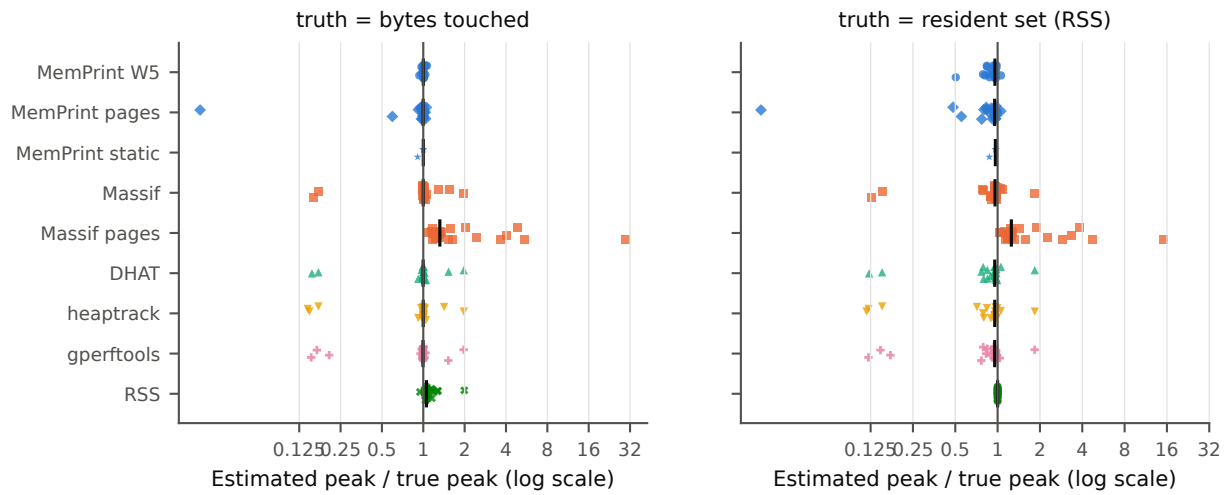


Figure 1: Each method's peak over the true peak, every program at its largest input (one marker per program, the black bar is the median). Left: against bytes touched. Right: against the RSS.

llama2.c at 0.02 and llama.cpp at 0.60. Massif with `-pages-as-heap` overestimates every program (1.07 to 29.9 times), and the RSS is 0.95 to 1.99 times the bytes touched (median 1.05).

4.2 Shape over time

The footprint of half of the programs varies over the run. The mean over the run is below 70% of the peak for 11 of the 25 programs (GCC's cc1, the four GAP kernels, Lua, LULESH, miniVite, Perl, Python and SQLite), and it is 0.36 to 0.40 times the peak for the four GAP kernels. A scheduler that holds the peak for such a job reserves memory that the job leaves unused for most of its run. Figure 2 shows Python's footprint, which rises and falls seven times. MemPrint windowed at 5% follows it within a few percent. Massif's curve, on its clock of instructions, follows its phases but sits above it, because Massif counts requested heap bytes and Python allocates memory it never touches (1.29 times the true peak). DHAT records only the peak, its time and the heap at exit, so a curve built from those points misses every phase but one.

The LLM engines differ from the rest of the corpus in where their memory is (Figure 3). Both map the model's weights from the file, and both read all of them for the first token, so the true footprint reaches 90% of its peak within the first 1 to 2% of a llama2.c run and stays there (mean 0.96 to 0.98 of the peak). llama.cpp reaches it at 12% of a 128-token run, and its mean is 0.89 of the peak. The heap profilers see only the heap: 0.08 to 0.21 of llama2.c's peak, and the KV cache and compute buffers of llama.cpp. MemPrint windowed at 5% reaches about the right peak (0.86 to 1.00 over the five configurations), but late: its curve reaches 90% of the peak at 19 to 20% of a llama2.c run and at 56% of the llama.cpp run, because between its windows it tracks heap blocks and writable pages, not a read-only file mapping, so it learns of the weights only when a window watches the program reading them. Pages only, which never watches, reads llama2.c at 0.02 to 0.05 of its peak.

4.3 Cost

MemPrint's windowed estimates are not cheaper than the heap profilers on these short runs (Figure 4). At the largest inputs, the median slowdown is 1.4× for heaptrack, 1.2× for gperftools, 5.7×

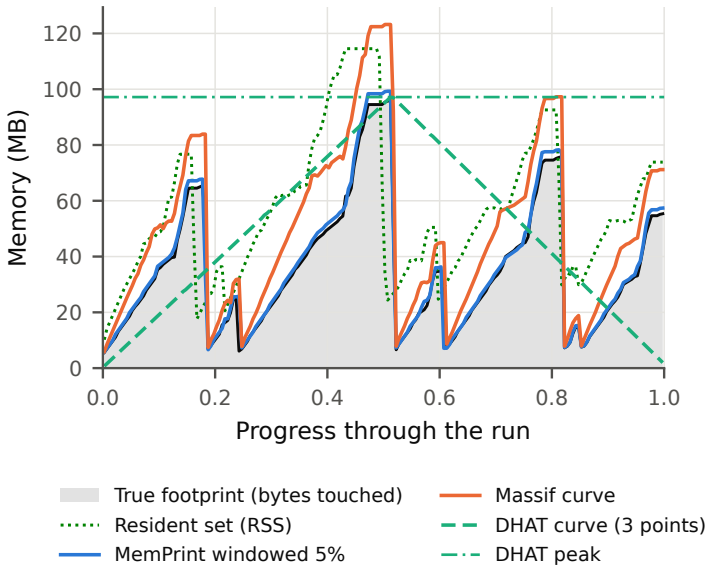


Figure 2: Python (largest input): the true footprint over the run, the RSS, and the curves and peaks the estimators report. Each curve is on its own method's clock, rescaled to progress.

for Massif (6.4× with its clock in milliseconds) and 23.8× for DHAT, against 6.3× for MemPrint pages only, 8.9× at 1% watched and 11.4× at 5% watched. The full Pin trace costs 226.8× (41.2 to 772.5×). The two LLM engines sit at the ends of that range. llama2.c, a single-threaded run of 2 to 15 s, costs 41 to 49× under the full trace, 3.5 to 6.6× under MemPrint at 5% and 19 to 21× under DHAT. llama.cpp, a sub-second run with four threads, costs 469 to 604× under the full trace, 47 to 91× under MemPrint at 5%, 82 to 137× under Massif and 182 to 280× under DHAT, while heaptrack and gperftools stay at 2.4 to 5.8×. Pin's fixed start-up cost weighs heavily on runs of a few seconds, and the windowed mode is slowest on programs with much code to re-translate (GCC's cc1, 93.6× at 1%). The static route needs no run of the program, but its interpreter takes 7 to 59,000 times the native run. It predicts atax MEDIUM in 0.8 s, the other PolyBench kernels at MEDIUM in 13 to 24 s and HPCCG in 255 to 989 s, but NPB IS and CG at class W take 4.0 and 5.5 hours.

4.4 Static predictions

The static predictions are within 0.46% of the true peak at the median, over the 19 (program, input) pairs they cover. They fail in two known ways. For LULESH, the static route predicts the footprint of the whole run, 1.57 times the live peak, because LULESH frees and reallocates its arrays between iterations and the static route does not model frees for C++ programs. For NPB MG, the prediction is 7 to 9% short, as it was in the blind test, for a reason we have not found.

5 Results

We first ask how much an exact footprint is worth and where the gain comes from, then how close each measured estimator gets, and finally what each estimator's cost buys. Table 3 gives every oracle at offered load 0.8 with calibrated 90% margins, and Figure 5 gives the response time across loads.

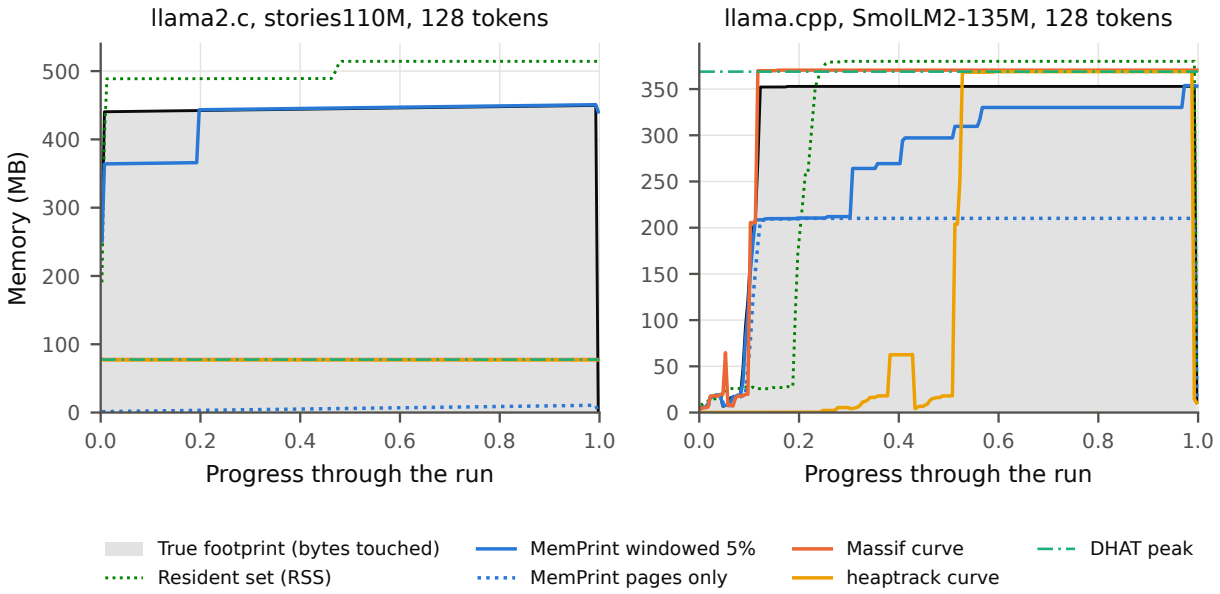


Figure 3: The two LLM engines at their largest inputs: the true footprint over the run, the RSS, and the curves and peaks the estimators report, each on its own clock rescaled to progress.

5.1 An exact footprint curve

Knowing a job's footprint curve, rather than its peak, is where most of the gain lies. At load 0.8 the exact curve gives a mean response time of 84 minutes, against 448 minutes for the exact peak and 2,208 minutes for the user's request, under the bytes-touched truth, which makes the curve 26.4× better than the request and 5.3× better than the peak, and the peak 4.9× better than the request. The RSS truth gives the same picture (99, 423 and 2,183 minutes). The curve lets the scheduler keep 81% of all memory resident with jobs, against 58% for the exact peak and 27% for the request, because jobs whose peaks fall at different times share the memory that a peak would hold idle. Coarsening the exact curve to the 20 widened windows used for every measured curve raises the response to 173 minutes, so about three quarters of the curve's advantage over the peak survives the resolution and the timing slack a measured curve needs. The gap widens with load (Figure 5). At load 0.4 every oracle that does not over-reserve keeps up with arrivals, and from load 0.6 the peak and the request saturate memory while the curve does not.

5.2 Measured estimators

MemPrint windowed at 5% is the only measured estimate that beats the exact peak, and only under the bytes-touched truth. With calibrated 90% margins it gives 389 minutes, against 448 minutes for the exact peak, 469 for MemPrint pages only, 580 for reactive resizing, 765 for MemPrint windowed at 1%, and 985 and 1,011 minutes for the RSS and DHAT peaks. The LLM engines cost the other two MemPrint curves their place: without them, pages only and windowed at 1% gave 381 and 358 minutes, but their curves lag the weights the engines read at the start (Section 4), and the margin that covers 90% of the programs rises from 4 to 8% for pages only and from 5 to 14% for windowed at 1% (Table 4), with 1,117 to 1,901 OOM kills per stream. The MemPrint 5% curve still loses to the exact curve by a factor of 4.6, for two reasons. First, the 20-window coarsening alone costs a factor of 2.1 (from 84 to 173 minutes). Second, a calibrated 90% margin leaves the

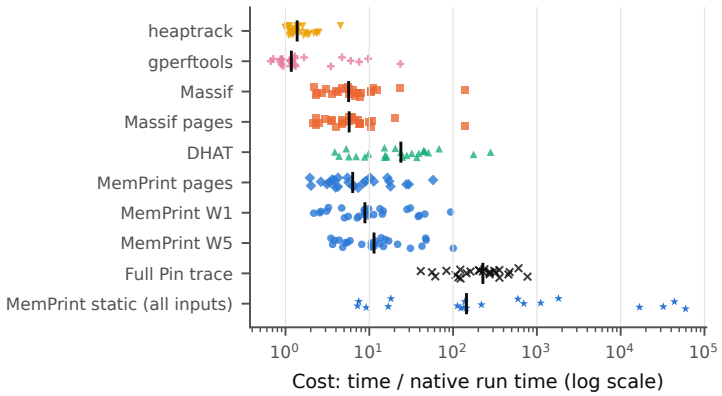


Figure 4: Each method's time over the native run, every program at its largest input (static: every input with a prediction). The black bar is the median.

programs that need more than the 90th-percentile margin under-reserved, which causes 461 OOM kills per stream. With a fixed 10% margin it gives 470 minutes and 216 kills.

Massif reports the heap over the whole run, not only its peak, and its curve follows a program's phases on its default clock of instructions executed. With calibrated 90% margins it gives 2,399 minutes, worse than the user's request, because the headroom that covers 90% of the programs is 52% of its peak (Table 4). Three kinds of program set that headroom. Massif reads GCC's cc1 at 0.16 times its peak, because the compiler's garbage collector maps its own pages, and llama2.c at 0.17, because its weights are a file mapping, and on the four GAP kernels its clock of instructions drifts from the truth's clock of memory references enough to need 12 to 53% headroom. With a fixed 10% margin, which leaves those jobs to be killed (1,714 kills per stream), Massif gives 991 minutes, close to DHAT. The clock matters. With Massif's clock in milliseconds under Valgrind, which includes Valgrind's start-up and runs at a different pace from the native program, the headroom rises to 100% and the response time to 4,268 minutes. On atax, for example, the heap reaches a tenth of its peak at 48% of Massif's millisecond clock but at the start of the native run. Massif with `-pages-as-heap` counts mapped pages, 1.32 times the true peak at the median, and gives 4,670 minutes. The heap profilers' peaks (DHAT, gperftools) are within 5% of the true peak for 20 of the 25 programs, so they come close to the exact peak for most jobs, but they read GCC's cc1 at 0.14 to 0.16 times its true peak and llama2.c at 0.17, and those jobs are killed and restarted repeatedly. For DHAT, these two programs fall above the 90th percentile of the margins, so its calibrated margin stays at 6% and it gives 1,011 minutes with 1,323 kills per stream. For gperftools and heaptrack a third program falls there as well, Python, whose heap they read at 0.21 and 0.14 times its peak, so the margin that covers 90% of the programs becomes 233% and 326% of the estimate, every job reserves three to four times its estimate, and the response time rises to 10,476 and 11,778 minutes, about five times the request's. With a fixed 10% margin, which accepts those kills, they give 1,158 and 1,154 minutes. The ML peak (1,991 minutes) improves little on the request it scales, because its 0.95-quantile prediction of u covers only 76% of jobs from unseen users. Reactive resizing (580 minutes) is the best oracle that needs no profile, although it is shown its future resizes when the job is admitted.

Pond and dynamic provisioning. Dynamic provisioning gives 457 minutes under the bytes-touched truth, between MemPrint windowed at 5% (389) and the exact peak (448), and 395 minutes under the RSS truth, where it beats every oracle but the exact curve, the exact peak included (423).

Table 3: Every oracle at offered load 0.8, calibrated 90% margins (10% for the request, ML and reactive oracles, which have no profile to calibrate; none for Pond and dynamic provisioning, whose Kills column counts failed resizes), mean of three seeds. Resp. is the mean response time, BSLD the mean bounded slowdown, Kills the OOM kills per stream of 3,000 jobs, Res. and Used the memory reserved and actually resident as a share of all memory.

	Truth = bytes touched					Truth = RSS				
Oracle	Resp. (min)	BSLD	Kills	Res. %	Used %	Resp. (min)	BSLD	Kills	Res. %	Used %
Exact curve	84	2.4	0	81	81	99	3.0	0	80	80
Exact, 20 windows	173	6.3	0	80	72	160	5.7	0	80	73
MemPrint windowed 5%	389	15.7	461	77	58	1869	80.5	309	58	33
Exact peak	448	18.3	0	78	58	423	17.2	0	77	61
Dynamic provisioning (5 min)	457	18.9	307	74	63	395	16.1	265	75	64
MemPrint pages only	469	19.2	1901	74	54	2516	109.0	1494	52	26
Reactive resizing (5 min)	580	24.2	19	71	54	612	25.6	21	70	54
MemPrint windowed 1%	765	32.2	1117	66	43	2340	101.3	620	54	28
MemPrint windowed 5%, peak	818	34.4	379	72	49	959	40.6	492	71	47
RSS peak (earlier run)	985	41.8	474	69	44	423	17.2	0	77	61
DHAT peak	1011	42.9	1323	70	45	1302	55.6	1310	65	41
ML peak (quantile GBM)	1991	86.0	115	61	29	1992	86.1	115	61	31
User request	2208	95.6	0	57	27	2183	94.4	0	57	29
Massif curve	2399	103.8	1107	53	26	2099	90.6	1184	56	30
Pond (untouched in pool)	2519	109.0	0	53	26	2472	106.9	0	53	27
DHAT curve (3 points)	3512	152.9	1228	51	23	3008	130.7	1157	52	25
DHAT hold-then-decay	4161	180.9	1144	48	20	3943	171.4	1152	47	21
Massif curve, ms clock	4268	185.9	1056	45	17	2684	116.3	1190	51	26
Massif pages-as-heap	4670	203.7	668	40	15	4472	194.4	692	42	16
Massif pages-as-heap, ms clock	6619	290.0	465	35	10	5669	247.5	697	37	12
gperftools peak	10476	458.4	1644	33	7	11181	490.8	1644	33	7
heaptrack curve	11778	516.0	1285	31	6	12610	552.6	1439	31	6

It pays with 307 and 265 failed resizes per stream, each a restart from the beginning. Two things make it this good, and both come from its simulation rather than from what a running system could know. Each resize is the largest use of the coming five minutes, read from the truth, so between resizes it is an exact curve at a five-minute resolution, with no margin and on the wall clock. And a job's first five minutes, at its full request, are a small share of runs with a median of 30 minutes. The reactive oracle, which resizes to the past five minutes instead, gives 580 and 612 minutes. Dynamic provisioning therefore bounds what a runtime monitor can reach; MemPrint's curve under the bytes-touched truth gets there before the job starts and without restarts. Pond gives 2,519 minutes, worse than the request (2,208), and causes no kills. It reserves the whole request, so it cannot pack more jobs than the request does, and the share it places in the pool must fit in the pool even when the node's local memory is free. With a pool of 4 TB instead of 1 TB, Pond and the request give 302 and 307 minutes (one seed, mean over all jobs). Its model predicts 8% of the request as untouched against 38% in truth, at the 0.98 quantile, and still overpredicts for 11% of the jobs, because they come from users the model has not seen. Pond saves local DRAM, not total memory, which is the resource the job streams contend for.

5.3 The RSS truth and the time axis

Under the RSS truth, the measured curves lose their advantage. MemPrint's windowed and pages-only curves give 1,869 to 2,516 minutes, worse than the DHAT peak (1,302 minutes) and the MemPrint 5% peak (959 minutes). The reason is the margin, not the shape. MemPrint's curve is on a clock of memory references, while the RSS truth is on wall-clock seconds, so a phase falls at a slightly different point of progress on the two curves, and MemPrint counts bytes rather than

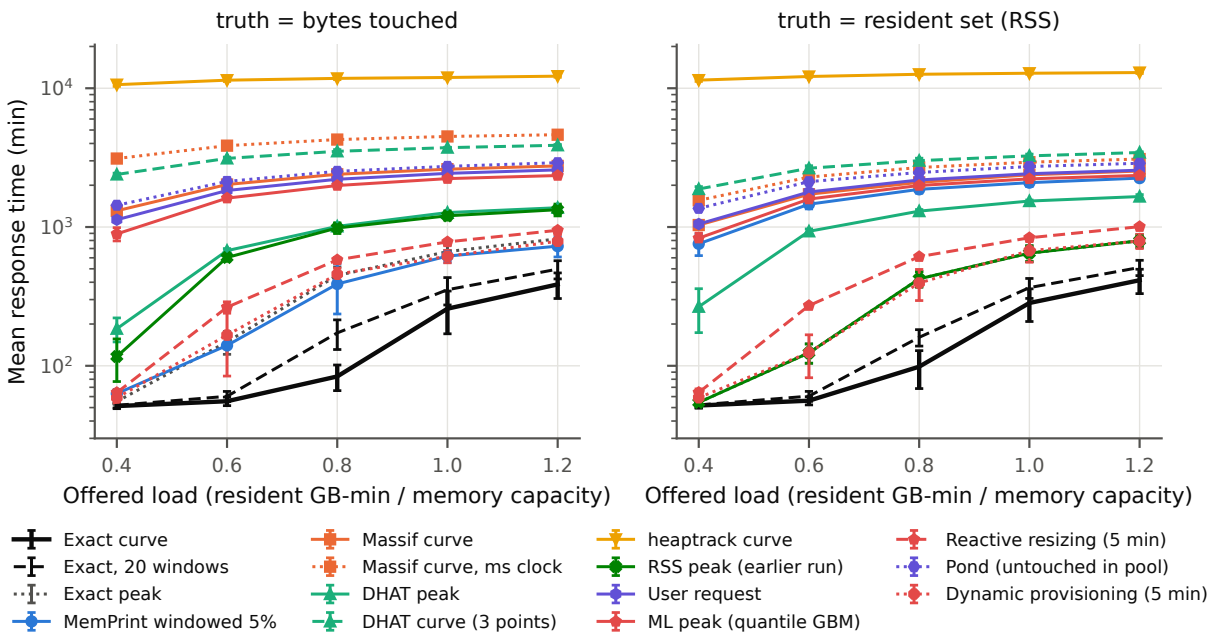


Figure 5: Mean response time against offered load, calibrated 90% margins (95% confidence intervals over three seeds, log scale). Left: the truth is bytes touched. Right: the truth is the RSS.

pages, which is half of miniVite's RSS. The margin that covers 90% of the programs is therefore 57% of the peak for MemPrint at 5%, against 6% under the bytes-touched truth (Table 4), and a curve with 57% headroom on its peak reserves more than a peak. A curve oracle is thus only as useful as its time axis is faithful to wall-clock time. Under the bytes-touched truth the MemPrint curves share the truth's clock of memory references, which favours them, and the RSS truth is the closer model of a real deployment. MemPrint does not record the native time of each snapshot, and Section 5.8 measures what placing its curve on the native clock does.

5.4 Margins

No margin policy rescues an estimator that is badly wrong for some programs (Figure 6). A small margin lets those programs' jobs be killed, and a margin large enough to cover them over-reserves every other job. With calibrated 100% margins, every heap profiler must cover GCC's cc1, whose peak it reads at 0.14 to 0.16, so each job reserves about 6.5 times its estimate and the response time rises to 15,000 to 17,000 minutes. Before the LLM engines joined the corpus, the MemPrint curves needed at most 7 to 19% under the bytes-touched truth; their late view of the mapped weights raises that to 40% at 5% watched, 136% at 1% and 41 times the estimate for pages only (llama2.c), so with calibrated 100% margins they give 1,299, 4,984 and 9,440 minutes. Taking each oracle at its best margin policy changes the order little (Figure 13 in the appendix): under the bytes-touched truth, MemPrint windowed at 5% gives 389 minutes, MemPrint pages only 469, Massif 991 (1,644 with its millisecond clock), DHAT 1,011 and heaptrack and gperftools 1,154 and 1,158.

5.5 Footprints predicted from source

The static route predicts nine of the programs (PolyBench 2mm, gemm and jacobi-2d at MEDIUM and atax at LARGE, HPCCG 64, LULESH 30 and NPB IS, CG and MG at class W), and we

Table 4: The margin each oracle needs to stay above the truth, as a share of its own peak (%), over the 25 programs at their largest inputs.

	Truth = bytes touched			Truth = RSS		
Oracle	Median	90th pct.	Max	Median	90th pct.	Max
MemPrint windowed 1%	0	14	136	6	69	150
MemPrint windowed 5%	0	6	40	6	57	112
MemPrint pages only	0	8	4112	7	79	4737
Massif curve	0	52	534	2	56	689
Massif pages-as-heap	0	28	62	0	30	63
Massif curve, ms clock	0	100	535	20	71	690
Massif pages-as-heap, ms clock	0	65	78	0	52	76
heaptrack curve	1	326	593	2	376	747
DHAT peak	0	6	546	2	24	706
DHAT curve (3 points)	36	90	580	53	99	739
DHAT hold-then-decay	21	90	580	39	99	739
gperftools peak	0	233	554	2	290	716
RSS peak (earlier run)	0	1	7	0	0	0

replay a stream made of those programs alone (Figure 7). Their footprints are flatter than those of the full corpus, so the exact peak comes close to the exact curve (273 against 194 minutes at load 0.8 under the bytes-touched truth). With calibrated 90% margins, the static predictions give 642 minutes, against 471 minutes for MemPrint windowed at 5%, 623 for heaptrack, 642 for Massif, 662 for the DHAT peak and 2,038 for the request, a 3.2× gain over the request without running the program. Two predictions cause most of the static oracle's 343 kills per stream: NPB MG is 7 to 9% short, and the margin calibrated on the other eight programs is 0%, which does not cover it. LULESH is over-predicted by 1.57 times, which wastes memory rather than causing kills. Under the RSS truth the static curve, like the windowed one, needs more headroom, and it gives 1,709 minutes against 1,399 for the DHAT peak.

5.6 LLM inference jobs

To see how the estimators fare on a kind of job whose memory is mostly read-only model weights, we replay a stream made of the LLM engines alone (Figure 8, Table 5). Each job takes the footprint and the estimates of one of the five traced configurations (llama2.c with the 15M, 42M and 110M models, and llama.cpp with 32 and 128 tokens), and each configuration's calibrated margin comes from the other four, as a site that has run other models or lengths of the same engines would set it.

These footprints are flat, so the curve gains less over the peak than in the full corpus: at load 0.8 the exact curve gives 180 minutes, the exact peak 307 and the request 2,067 under the bytes-touched truth, a 1.7× gain of the curve over the peak against 5.3× for the full corpus. The best measured estimate is the RSS peak of an earlier run, 598 minutes with no kills under the bytes-touched truth and equal to the exact peak under the RSS truth, because the weights the RSS counts are the footprint. Reactive resizing gives 753 minutes and MemPrint's 5% peak 785 minutes, 602 kills per stream. Dynamic provisioning, which reads the next five minutes of the truth, gives 386 and 443 minutes with 140 and 200 restarts, and Pond 2,379 and 2,450.

Every heap profiler fails. They read llama2.c at 0.08 to 0.21 of its peak and llama.cpp at about

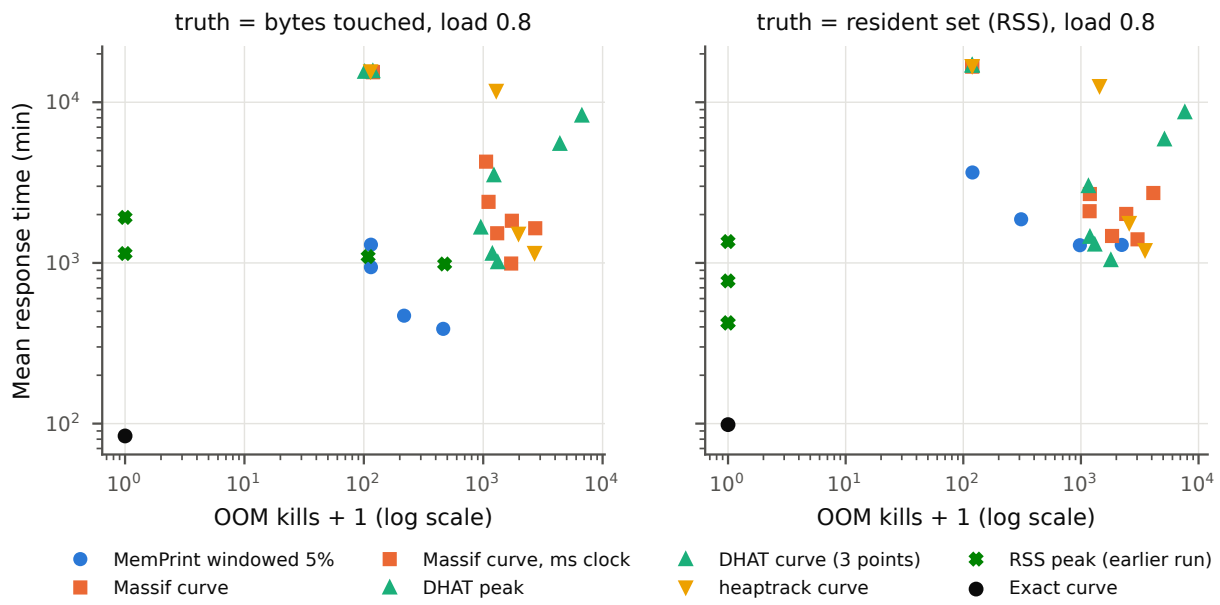


Figure 6: Mean response time against OOM kills at load 0.8, one marker per margin policy (10%, 30%, calibrated 90% and 100%). No margin gives a measured estimator both few kills and a short response time, except the MemPrint curves under the bytes-touched truth.

1.05, so the margin calibrated on the other configurations cannot cover llama2.c's jobs, which would need headroom of 3.8 to 10.6 times the estimate: with calibrated margins Massif, DHAT, heaptrack and gperftools give 8,747 to 9,344 minutes and 1,195 kills per stream, four times the request. With a fixed 10% margin, llama2.c's jobs are killed until the requeue factor has raised their reservation enough, 9,050 to 9,647 times per stream for DHAT, Massif and gperftools (15,661 for heaptrack), and their response of 613 to 823 minutes hides that waste. Massif with `-pages-as-heap` counts the file mapping and never under-reads, so it is never killed, but it over-reserves (2,650 minutes).

MemPrint's curves fail for a different reason. Its peaks are about right (0.86 to 1.00 of the true peak at 5% watched), but its curves reach the peak late (Section 4), while every job's footprint jumps to the weights in the first percent of its run. A curve reservation is therefore too small at the start of every job, and the configurations need 17 to 96% headroom at 5% watched; the calibrated margin covers some and not others, and the curves give 3,120 to 6,685 minutes with 610 to 1,049 kills per stream. Pages only, which never watches, needs headroom of 20 to 41 times its estimate for llama2.c. This is the same finding as under the RSS truth, from another cause: a curve is worth only as much as its timing, and MemPrint's windowed timing misses memory that a program maps and reads before the first window. Section 5.7 adds file mappings to what the windowed mode tracks between windows.

5.7 Tracking file mappings between windows

Between windows, MemPrint's windowed mode tracks the program's heap blocks and anonymous mappings and reads their resident pages from `/proc/self/pagemap` at every snapshot, but a file mapping was not a block, so its pages were seen only through the samples a window took. We added the program's file mappings to the tracked memory (`-file_maps`, now the default). The tool catches every `mmap` of a file without `PROT_EXEC` at the system call, trims the mapping on `munmap` or when a later mapping replaces part of it, and leaves out mappings inside a loaded image's address

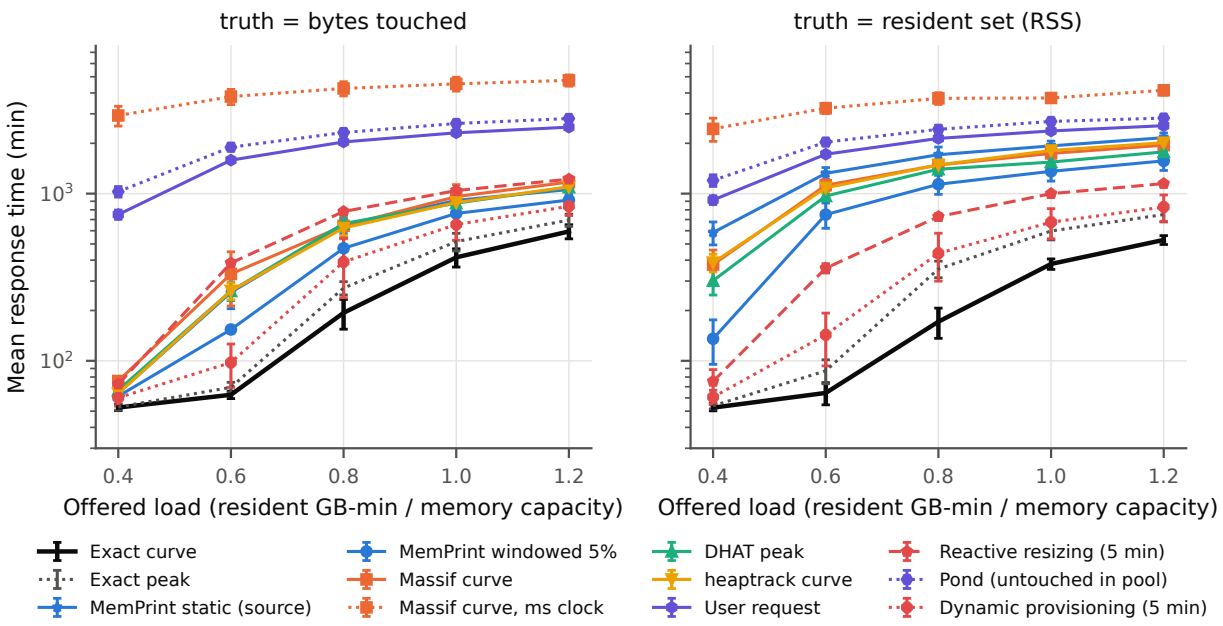


Figure 7: The nine programs the static route predicts (each at its largest input with a prediction): mean response time against offered load, calibrated 90% margins.

range, which are the loader's mappings of a library whose writable sections are already counted. At every snapshot the resident bytes of the remaining file mappings are added to the estimate, as those of a fresh heap block are, and the windows' samples that fall in them are not counted a second time. A test program that maps a 4 MB file and reads it before its first window is estimated at 4.1 MB at its peak with the option and at 0.14 MB without it.

We repeated MemPrint's windowed and pages-only runs at every program's largest input and at every LLM input with the new tool, and replayed both streams (Table 6). Only the LLM engines change materially: their peaks become 1.00 for every mode (llama2.c pages only: from 0.02), and the curves reach 90% of the peak at the same point of the run as the truth. Perl and SQLite, the only other programs whose peak moves, change by 1%. The margin that covers 90% of the programs falls to 1% at 5% watched, 4% at 1% and 4% for pages only (from 6, 14 and 8%), and in the LLM stream to 1% for every mode (from 92%, 137% and 37 times the estimate). On the main stream under the bytes-touched truth, all three MemPrint curves now beat the exact peak (448 minutes): windowed at 5% gives 327 minutes (389 before), at 1% 405 (765) and pages only 411 (469), and windowed at 5% needs 160 GB per node for the target service level, as the exact peak does, instead of 192 GB. On the LLM stream the curves give 365 to 386 minutes instead of 3,120 to 6,685, better than the earlier run's RSS peak (598) and close to the exact peak (307); the remaining 597 kills per stream come from llama2.c with the 15M model, which needs 1.2 to 1.4% of headroom, more than the margin calibrated on the other four configurations (under 0.5%). The option does not change the time axis, so under the RSS truth the curves still need their wall-clock headroom: 1,843 to 1,873 minutes on the main stream and 930 to 948 on the LLM stream, against 423 and 361 for the RSS peak.

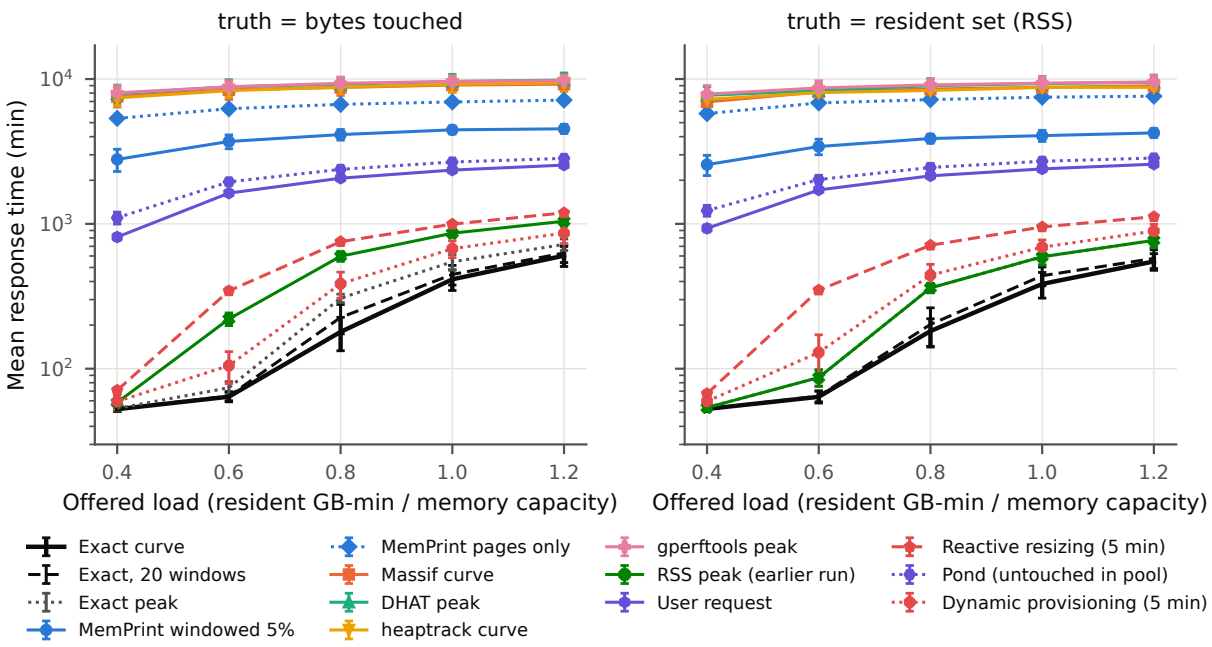


Figure 8: A stream of LLM inference jobs alone (llama2.c with three models, llama.cpp with two lengths): mean response time against offered load, calibrated 90% margins, each configuration's margin calibrated on the other four.

5.8 The time axis and the level

Under the RSS truth, MemPrint's curves needed about 57% headroom, and we attributed it to their clock. Three changes close most of that gap (Table 7, Figure 9).

The native clock. The time under Pin is not the native time, because Pin slows a program's phases unevenly: GAP's graph construction, which holds its peak, takes a fifth of the native run but a twentieth of the run under Pin. We therefore measure the native clock: one more native run under `perf stat -I 5` counts the user-level loads and stores every 5 ms, which gives the native time at which each share of the run's memory references has executed, and a MemPrint sample taken after a share f of the references is placed at that time. No instrumentation is involved, and the run costs what the program costs. On the native clock the margin that covers 90% of the programs falls from 57% to 24%, and the response time from 1,873 to 694 minutes.

The level. The RSS counts memory MemPrint does not: code, the allocator's chunk headers and the memory it keeps after a free. A hybrid oracle keeps the shape of MemPrint's curve on the native clock and scales it so its peak is the peak of an earlier run's RSS, so the curve knows when and the RSS how much. Its margin falls to 17%, and it gives 609 minutes at 5% watched and 569 for pages only. With a fixed 10% margin it gives 415 to 417 minutes against 423 for the exact peak, with 313 to 316 kills per stream.

The code. The RSS truth scales a program's whole RSS above its smallest sample with the job's size, so the code of the program and of its libraries, which does not grow with the job's data, is scaled as if it did. A second native run with the sampler reading each executable mapping's resident bytes from `smaps` measures the code: 1 to 27 MB, 10.4 of miniVite's 30.6 MB, 19.4 of cc1's 134 and 27 of llama2.c's 514. With the code left out of the truth (and of the earlier run's RSS), the hybrid with pages only gives 422 minutes against 429 for the exact peak. These are the first

Table 5: The LLM stream at offered load 0.8, calibrated 90% margins (10% for the request, ML and reactive oracles, none for Pond and dynamic provisioning), mean of three seeds; columns as in Table 3.

	Truth = bytes touched					Truth = RSS				
Oracle	Resp. (min)	BSLD	Kills	Res. %	Used %	Resp. (min)	BSLD	Kills	Res. %	Used %
Exact curve	180	6.6	0	76	76	182	6.7	0	74	74
Exact, 20 windows	226	8.6	0	75	72	202	7.5	0	76	73
Exact peak	307	12.1	0	75	69	361	14.4	0	76	66
Dynamic provisioning (5 min)	386	15.8	140	72	65	443	18.3	200	71	63
RSS peak (earlier run)	598	24.9	0	69	57	361	14.4	0	76	66
Reactive resizing (5 min)	753	31.9	10	65	54	712	29.9	9	66	54
MemPrint windowed 5%, peak	785	32.9	602	68	51	913	38.4	602	68	46
ML peak (quantile GBM)	1836	79.1	115	61	36	1904	82.1	115	61	34
User request	2067	89.3	0	58	33	2146	92.7	0	57	31
Pond (untouched in pool)	2379	102.8	0	52	30	2450	105.9	0	53	28
Massif pages-as-heap	2650	114.7	0	47	26	2186	94.5	0	52	30
Massif pages-as-heap, ms clock	2955	127.9	0	45	24	2495	107.9	0	48	26
MemPrint windowed 1%	3120	135.7	1049	46	24	3083	134.0	1046	47	23
MemPrint windowed 5%	4134	180.6	1034	43	19	3884	169.6	1033	44	19
MemPrint pages only	6685	293.2	610	29	9	7208	316.3	1220	30	9
Massif curve	8747	381.2	1195	32	9	8528	372.3	1195	32	10
heaptrack curve	8794	383.2	1195	32	9	8333	363.2	1195	32	10
Massif curve, ms clock	8869	386.7	1195	32	9	8630	376.5	1195	32	10
DHAT curve (3 points)	9103	397.6	1195	31	9	8776	383.4	1195	32	9
DHAT peak	9198	400.7	1195	31	9	8986	391.7	1195	32	9
gperftools peak	9344	408.0	1195	31	9	9108	397.5	1195	32	9
DHAT hold-then-decay	9561	417.3	1195	31	8	9251	404.3	1195	32	9

measured curves that keep up with the exact peak under a resident-set truth; plain MemPrint gives 1,920 minutes there.

What remains. No curve beats the peak under the RSS truths, as the exact curve does (89 to 99 minutes). Even on the native clock, the true footprint of GAP's construction holds its peak for one or two twentieths of the run, while the RSS holds its 76 MB for four, and Lua's RSS stays at 62 MB after its live data has fallen to 17 MB: the allocator keeps freed memory resident, which an estimate of live bytes cannot see. A curve of the RSS itself, taken from an earlier run, would see it, and is the next oracle to test.

On the LLM stream under the RSS truth, the hybrids give 400 to 417 minutes against 948 for plain MemPrint and 361 for the exact peak. In the memory sweep, MemPrint's native-clock curves meet the service level with 192 GB per node under both RSS truths (7.0 TB), instead of 256 GB on their clock of references; the exact peak needs 160 GB.

5.9 Memory needed for the same service

The response-time gains translate into memory a site would have to buy (Figure 10). At a fixed arrival rate, the exact curve meets the target with 128 GB per node (5.0 TB in all). Under the bytes-touched truth, the exact peak needs 160 GB per node (6.0 TB, 20% more), MemPrint's windowed and pages-only curves and reactive resizing 192 GB (7.0 TB, 40% more), DHAT, an earlier run's RSS and the ML peak 256 GB (9.0 TB, 80% more), and Massif's curve and the request 320 GB (11.0 TB, 120% more). gperftools, heaptrack, Massif's curve on its millisecond clock and the DHAT curve do not meet the target even at 320 GB per node. With the LLM engines in the corpus, MemPrint's curves need one step more than before (192 instead of 160 GB per node), and gperftools and heaptrack fall from 256 GB to never, through their calibrated margins. Under the RSS truth, the exact peak and the RSS of an earlier run need 6.0 TB, reactive resizing 7.0 TB, MemPrint windowed

Table 6: MemPrint's oracles before and after tracking file mappings between windows (`-file_maps`): mean response time (minutes) at load 0.8, calibrated 90% margins, mean of three seeds. Only MemPrint's windowed runs were repeated; every other oracle is unchanged.

	Main stream (25 programs)				LLM stream			
	Bytes touched		RSS		Bytes touched		RSS	
Oracle	before	after	before	after	before	after	before	after
MemPrint windowed 5%	389	327	1869	1873	4134	386	3884	948
MemPrint windowed 1%	765	405	2340	1855	3120	370	3083	930
MemPrint pages only	469	411	2516	1843	6685	365	7208	939
MemPrint windowed 5%, peak	818	817	959	958	785	601	913	986

Table 7: Mean response time (minutes) at load 0.8 under three truths, the main stream, calibrated 90% margins. MemPrint's runs track file mappings (Section 5.7). Under the RSS truths, an earlier run's RSS peak is the truth's own peak, and equals the exact peak.

Oracle	Truth = bytes touched	Truth = RSS	Truth = RSS without code
Exact curve	84	99	89
Exact peak	448	423	429
RSS peak (earlier run)	985	423	429
MemPrint windowed 5%	327	1,873	1,920
MemPrint 5%, native clock	1,549	694	564
MemPrint pages, native clock	1,561	783	667
MemPrint 5%, native clock, RSS level	2,053	609	469
MemPrint pages, native clock, RSS level	2,028	569	422
DHAT peak	1,011	1,302	1,225
Massif curve	2,399	2,099	2,264
heaptrack curve	11,778	12,610	12,410
gperftools peak	10,476	11,181	11,106
User request	2,208	2,183	2,202

at 5%, DHAT, Massif, the ML peak and the request 9.0 TB, and MemPrint pages only 11.0 TB, which repeats the finding that a curve on a reference clock loses its advantage against a wall-clock truth. Dynamic provisioning needs 7.0 TB under the bytes-touched truth and 6.0 TB under the RSS truth, like the exact peak. Pond does not meet the target at 320 GB per node under either truth, because the pool, not local memory, is what its placement fills.

5.10 What each estimator's cost buys

Figure 11 sets each estimator's cost against what it buys. Under the bytes-touched truth, the full Pin trace costs 227 times the native run and gives a 26.4× gain over the request as a curve, or 4.9× as a peak. MemPrint windowed at 5% costs 11 times the native run and gives 5.7×, about a fifth of the exact curve's gain at a twentieth of its cost; pages only and windowed at 1% cost 6 and 9 times and give 4.7× and 4.3×. heaptrack and gperftools cost under 1.5× and give 1.9×, DHAT costs 24× and gives 2.2×, and Massif's curve costs 5.7× and gives 2.2× (1.3× with its millisecond clock). An earlier run's RSS costs nothing extra and gives 2.2× under the bytes-touched truth,

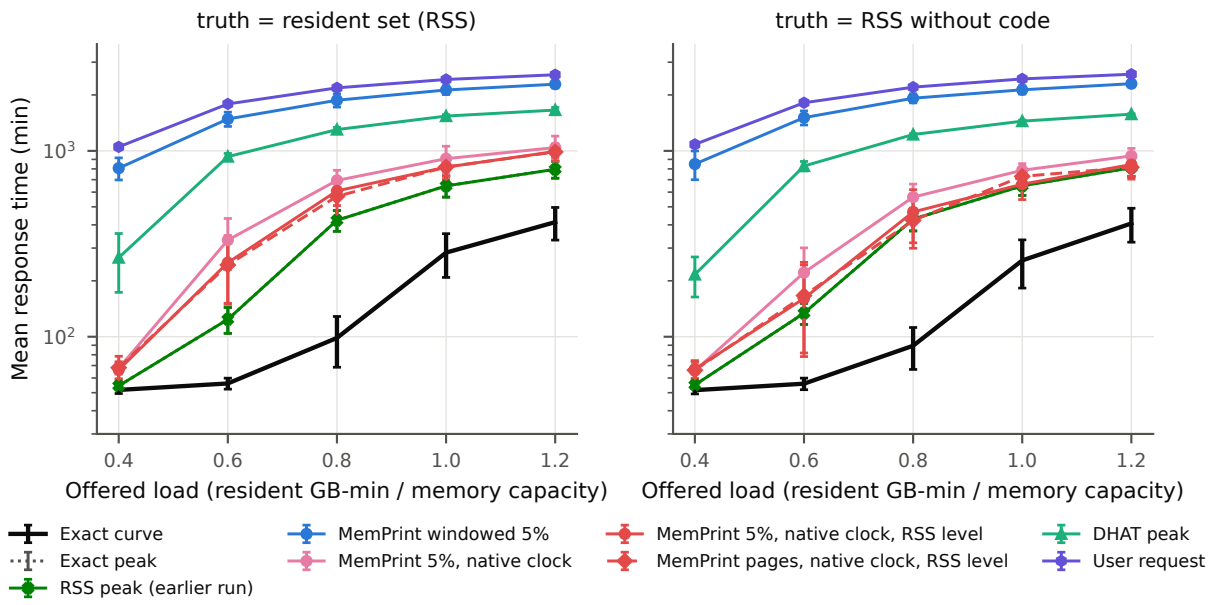


Figure 9: Mean response time against offered load under the RSS truth and the RSS truth without code, calibrated 90% margins: MemPrint's curve on its clock of references and on the native clock, and scaled to an earlier run's RSS peak. The exact peak lies under the RSS peak of an earlier run.

and under the RSS truth it is the truth's peak. Under the RSS truth the measured curves give 1.6 to 1.7× and the heap profilers' peaks 1.8 to 2.1×, so the profiling cost buys little until the curve's time axis is fixed.

6 DHAT curves

DHAT records the heap at its global peak (t-gmax), the time of that peak, and the heap at exit, but no time series [19]. A scheduler can use those three points in two ways besides the peak. The *DHAT curve* rises linearly from zero to the peak at its recorded time and falls linearly to the exit value. The *hold-then-decay* curve holds the peak from the start to its recorded time, which is safe before the peak, and falls linearly afterwards. Only "never above the peak" is guaranteed by what DHAT records, and that is the DHAT peak oracle.

Both DHAT curves do worse than the DHAT peak under every margin policy and both truths, except with calibrated 100% margins, where all three reserve for GCC's cc1 and give about the same (15,000 to 17,000 minutes; Figure 12). At load 0.8 with calibrated 90% margins, the DHAT curve gives 3,512 minutes and the hold-then-decay curve 4,161 minutes, against 1,011 minutes for the DHAT peak, under the bytes-touched truth, and 3,008, 3,943 and 1,302 minutes under the RSS truth. With fixed margins the curves are worse still (8,263 and 10,648 minutes at 10%), because they fall below the truth wherever a program has more than one phase, and their jobs are killed 5,751 to 7,642 times per stream. The median program needs 36% headroom on the DHAT curve (21% on hold-then-decay) and the 90th percentile needs 90% (Table 4). The average of the DHAT curve over the run is 0.81 times the true mean footprint at the median over the 71 (program, input) pairs, so it under-reserves on average as well as in places. DHAT's three points therefore do not substitute for a measured curve. The information that makes a curve useful to a scheduler is the timing of every phase, and DHAT records the timing of one.

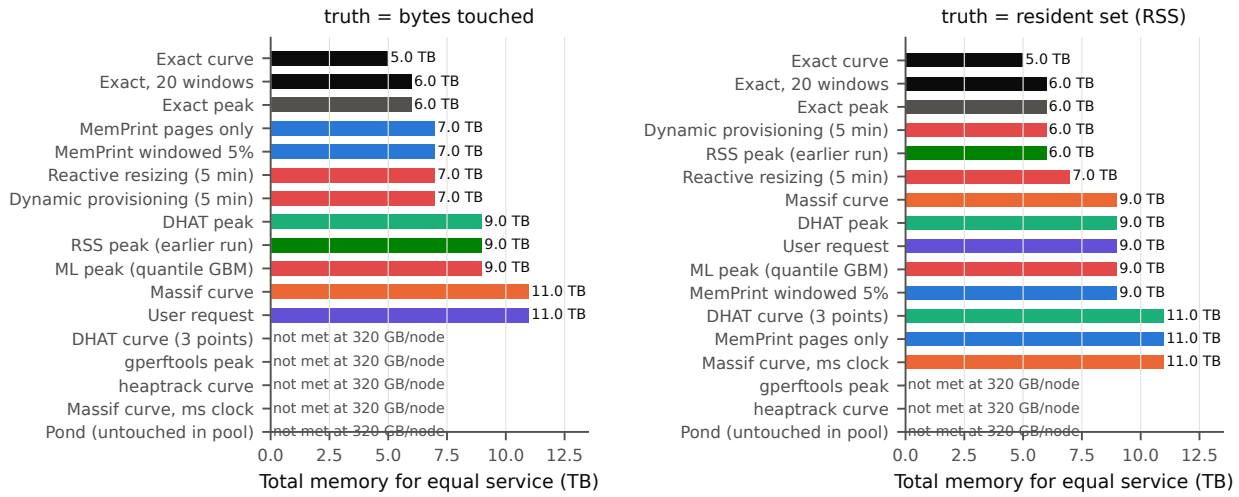


Figure 10: The total memory (32 nodes of local DRAM plus the 1 TB pool) each oracle needs to keep the mean bounded slowdown within 1.5 times that of the exact curve at 128 GB per node, at a fixed arrival rate (load 0.8 on the 128 GB cluster). Local memory per node was swept over 64, 96, 128, 160, 192, 256 and 320 GB.

7 Limitations

The results rest on a model of a cluster and of its jobs, and each simplification limits what they show.

Scaled footprints. The corpus programs peak at 15 to 450 MB, and the simulated jobs at tens to hundreds of GB. Scaling a measured curve to a job's size keeps its shape and every estimator's ratio to the truth, but it assumes the ratio does not change with the input. The corpus shows that it often does at small inputs: MemPrint windowed at 5% reads 0.61 to 0.92 times the peak of the PolyBench kernels at SMALL, and 1.00 at LARGE. We use the largest input of every program to keep this effect small.

Time axes. Every curve is put on the job's progress by linear rescaling of its own clock. The true footprint of a job over wall-clock time is known only for the RSS truth, so the bytes-touched truth favours the MemPrint curves, which share its clock of memory references (Section 5).

The job stream. The main stream is synthetic, with memory the contended resource by design, and the used-to-requested ratio comes from the LANL CM5 log of 1994 to 1996, the only archive log with both fields that we found. A real trace with requested and used memory per job would test whether the effect holds at the memory pressure of a production system.

Scheduler and machine. Jobs use one node, the slowdown is linear in the remote share of the footprint with one β for every job, the pool has no bandwidth contention, and only FCFS with EASY backfilling was tested. The OOM model requeues a killed job with 1.5 times its reservation, and a batch system that killed a job only once, or that swapped instead of killing, would change the kill counts.

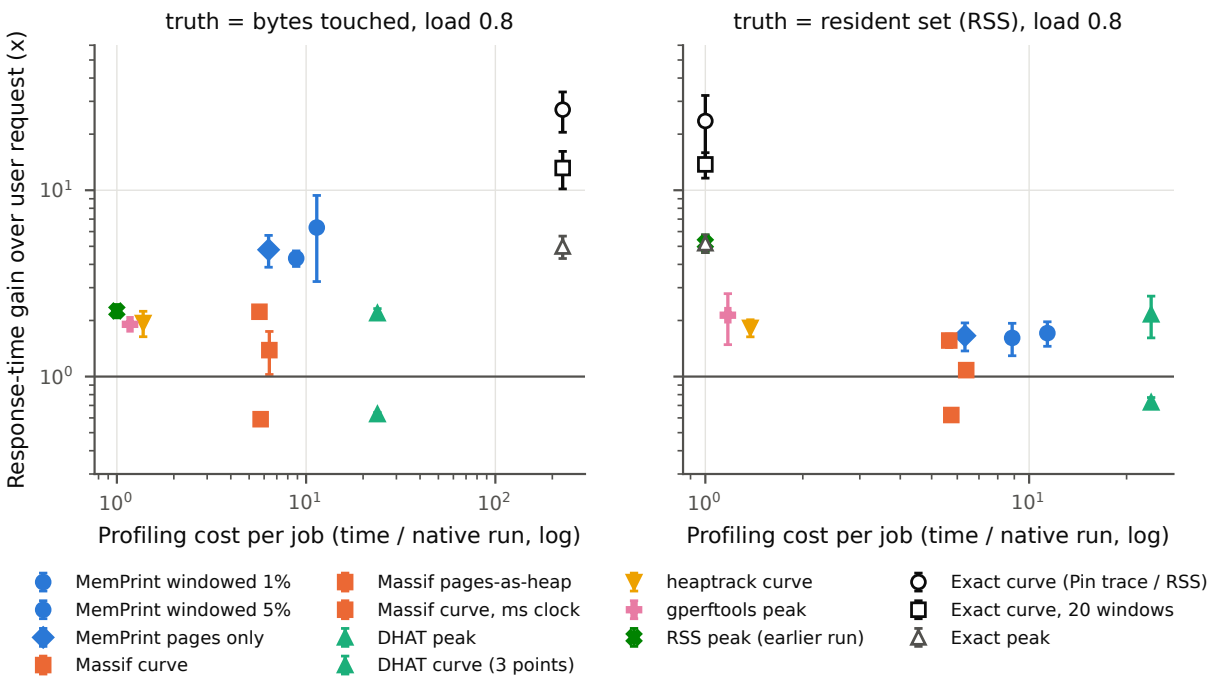


Figure 11: Profiling cost per job (median time over the native run, at the largest inputs) against the gain in mean response time over the user's request, at load 0.8, each oracle at its best margin policy. The exact curve's cost is the full Pin trace under the bytes-touched truth and an RSS sampler under the RSS truth.

Costs. The costs were measured with up to 15 runs sharing the node, so they are slowdowns under load, and Pin's fixed start-up cost weighs heavily on runs of a few seconds. Longer jobs would amortise it.

LLM inference. The LLM jobs are single completions of 32 to 128 tokens on the CPU, scaled like every other program. A serving engine that runs for hours fills its KV cache over many requests, and a GPU engine keeps its weights in device memory, so neither shape is covered here. llama.cpp's 360M model could not be traced, so the corpus has llama.cpp at two inputs of one model.

The static route. The static predictions cover 9 of the 25 programs. They predict a whole-run footprint for C++ programs, so a program that frees and reallocates its data, such as LULESH, is over-predicted, and the interpreter takes up to 5.5 hours for one input of NPB CG.

8 Conclusion

An exact footprint is worth most to a scheduler on disaggregated memory as a curve over the job's run, and the gain comes from letting jobs with peaks at different times share memory. On our corpus and streams, the exact curve gives a 26.4× shorter mean response time than the user's request and needs 5.0 TB where the request needs 11.0 TB. Heap profilers get most peaks right, but their large errors on a few programs, which come from memory that is mapped outside the heap or allocated and never touched, cannot be covered by any margin without over-reserving every other job; LLM engines, whose weights are a file mapping, add a third such program, which pushes the calibrated margins of heaptrack and gperftools past twice their estimates. Among measured

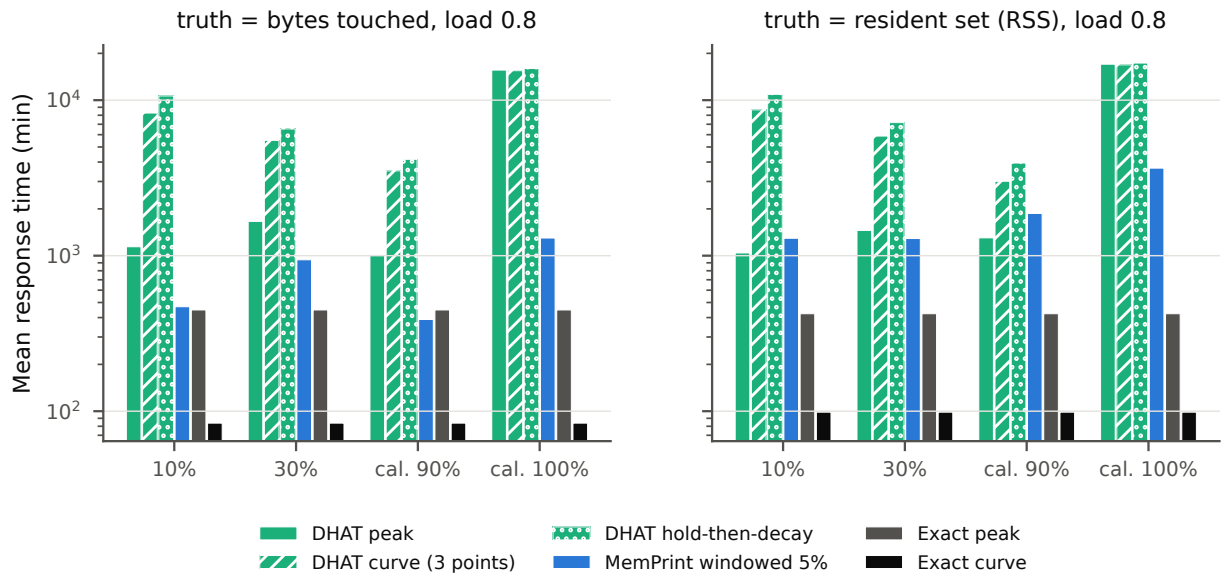


Figure 12: DHAT's peak and the two curves built from its three recorded points, against MemPrint windowed at 5% and the exact peak and curve, at load 0.8 under each margin policy (log scale).

estimators, MemPrint windowed at 5% comes closest to the exact curve, with the smallest margin (6% for 90% of programs) and 40% more memory than the exact curve for the same service, at 11 times the native run time. On LLM jobs alone it failed, because its curve learned of the weights only in its windows, while the footprint reaches its peak in the first percent of the run; tracking the program's file mappings between windows, as its heap blocks are tracked, removed that failure and made every MemPrint curve beat the exact peak on the full corpus. DHAT's three recorded points do not make a usable curve. Two results set the direction of further work. First, under the RSS truth every measured curve lost its advantage; placing MemPrint's curve on the native clock through hardware counts of memory references, at the level of an earlier run's RSS, brings it to the exact peak, but beating the peak there needs an estimate of the memory an allocator keeps after a free. Second, the static route predicts footprints without running the program, but its errors on programs that free and reallocate memory, and its cost on large inputs, limit it today to programs whose footprint is set by their input sizes.

A Additional figures

References

[1] Emmanuel Amaro et al. Can far memory improve job throughput? In *European Conference on Computer Systems (EuroSys)*, 2020.

[2] Robin Boezennec et al. Allocation strategies for disaggregated memory in HPC systems. In *IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2024.

[3] CXL-ClusterSim: Modeling CXL-based disaggregated memory cluster using gem5 and SST. arXiv:2605.27745, 2026.

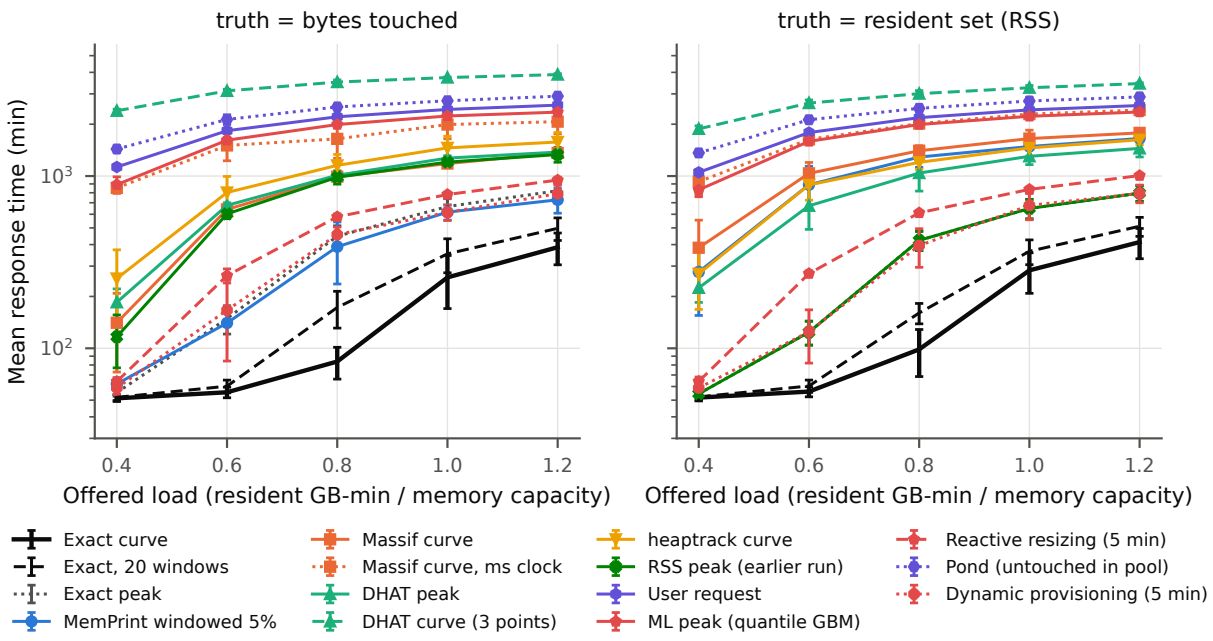


Figure 13: Mean response time against offered load, each oracle at its best margin policy at each load.

[4] CXL-DMSim: A full-system CXL disaggregated memory simulator. arXiv:2411.02282, 2024.

[5] Dror G. Feitelson, Dan Tsafrir, and David Krakov. Parallel workloads archive. `https://www.cs.huji.ac.il/labs/parallel/workload/`. Logs LANL CM5 (LANL-CM5-1994-4.1-cln) and MetaCentrum2 (METACENTRUM-2013-3).

[6] Georgi Gerganov and ggml contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, release b11500, 2026.

[7] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[8] gperftools: TCMalloc and heap profiler. `https://github.com/gperftools/gperftools`. Version 2.18.

[9] Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. `https://github.com/karpathy/llama2.c`, 2023.

[10] KS+: Predicting workflow task memory usage over time. arXiv:2408.12290, 2024.

[11] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[12] David A. Lifka. The ANL/IBM SP scheduling system. Job Scheduling Strategies for Parallel Processing (JSSPP), LNCS 949, 1995.

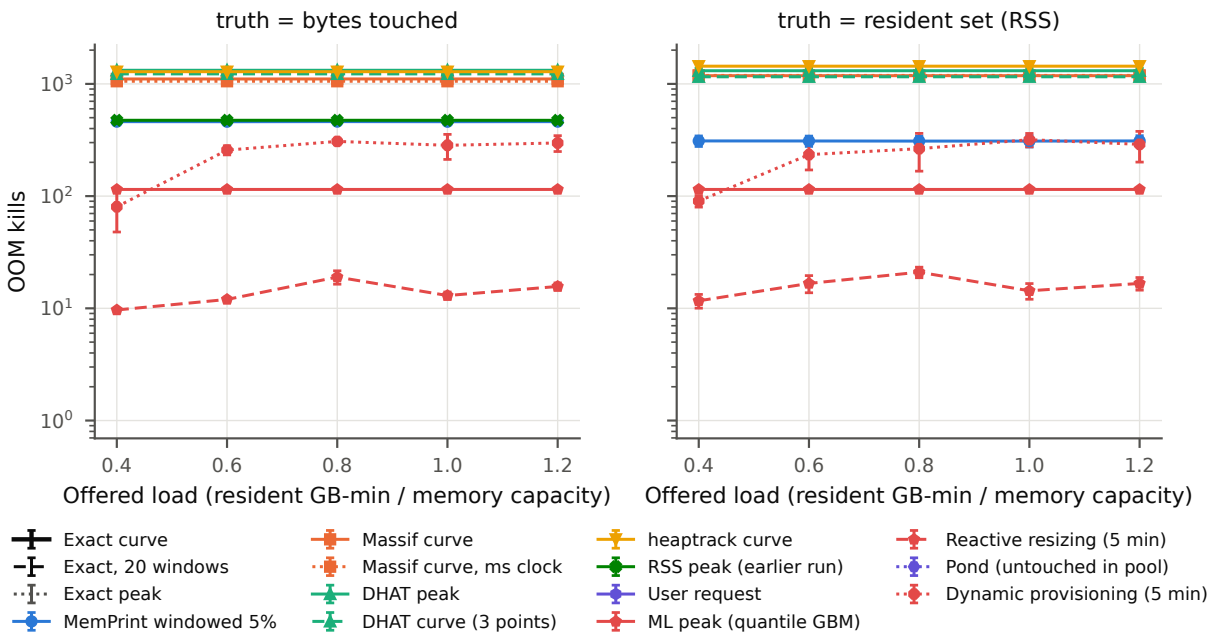


Figure 14: OOM kills per stream of 3,000 jobs against offered load, calibrated 90% margins.

[13] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[14] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[15] Ivy Peng et al. On the memory underutilization: Exploring disaggregated memory on HPC systems. In *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020.

[16] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. `https://sourceforge.net/projects/polybench/`.

[17] Amit Puri et al. DRackSim: Simulator for rack-scale memory disaggregation. In *ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2024. arXiv:2305.09977.

[18] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at `http://valgrind.org/docs/manual/ms-manual.html`.

[19] Valgrind Developers. *DHAT: a dynamic heap analysis tool.* Available at `https://valgrind.org/docs/manual/dh-manual.html`.

[20] Milian Wolff. heaptrack: a heap memory profiler for Linux. `https://github.com/KDE/heaptrack`. Version 1.3.0.

[21] Yiwei Yang et al. CXLMemSim: A pure software simulated CXL.mem. arXiv:2303.06153, 2023.

[22] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Improving HPC system throughput and response time using memory disaggregation. `https://upcommons.upc.edu/`, 2021.

[23] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2021.

[24] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.